

Examen machine - OS202

Parallélisation d'un code de simulation de galaxie

Réalisé par :

Théotime GALMICHE

Professeur référent :

Augustin Parret-Fréaud

ENST2



IP PARIS

Table des matières

1	Introduction	2
2	Question préliminaire	2
3	Mesure de temps initial	2
4	Parallélisation en numba du code séquentiel	2
5	Séparation affichage/calcul	3

1 Introduction

L'examen portera sur la simulation de l'évolution des étoiles dans une galaxie comprenant un trou noir massif de taille N . Pour cela, nous réaliserons tout d'abord une mesure initiale du temps, puis nous paralléliserons avec numba le code séquentiel.

Ensuite, nous séparerons l'affichage et le calcul, avant de paralléliser le calcul.

2 Question préliminaire

En lançant la première simulation avec les paramètres proposés en exemple `python3 nbodies_grid_numba.py data/galaxy_5000 0.0015 15 15 1`, nous observons que la galaxie s'étend en grande majorité sur 2 dimensions, celles couvertes par N_i et N_j . Ainsi, pour faciliter la représentation, et ne pas trop augmenter le nombre de cellules (puisque chaque incrémentation sur N_k double le nombre de cellules), le problème peut être modélisé comme un problème "2D" et ainsi on pourra fixer : $N_k = 1$.

3 Mesure de temps initial

Toujours avec les mêmes paramètres de simulation, nous observons :

$$Render\ time \approx 1\ ms, \quad Update\ time \approx 585\ ms$$

Nous pouvons donc conclure que c'est le temps de calcul (*Update time*) qui est le facteur limitant actuellement, et donc le plus intéressant à paralléliser.

4 Parallélisation en numba du code séquentiel

En appliquant la parallélisation numba avec `prange`, nous obtenons une accélération intéressante :

$$Speedup = \frac{T_{seq}}{T_{par}}$$

La différence est déjà très visible lorsqu'on lance la simulation. De nombreuses étoiles sont "éjectées" de la galaxie, et le trou noir prend de l'ampleur. Nous obtenons des mesures proches de 50 ms, soit un speedup de l'ordre d'un facteur ≈ 10 . Cependant, le nombre de threads n'influence que très peu le temps de calcul, comme le montre le graphe obtenu pour 1, 2, 4 et 8 threads :

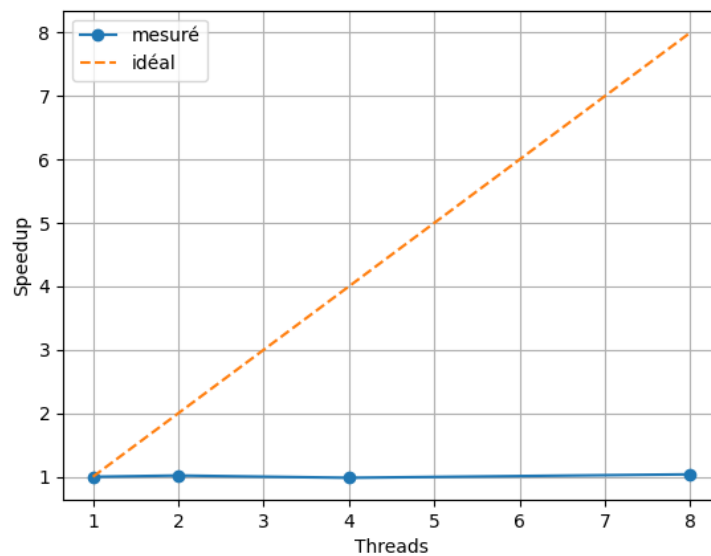


FIGURE 1 – Temps de calcul en fonction du nombre de threads

Cela pourrait être dû à des boucles trop petites, ou bien à des dépendances trop importante, qui limitent le gain.

5 Séparation affichage/calcul

La fonction `run` a été adaptée pour séparer l’affichage et le calcul grâce à MPI : le processus 0 se charge uniquement de l’affichage, tandis que le processus 1 réalise les calculs des positions. Les échanges de données entre les deux processus se font via `comm.send` et `comm.recv`, en utilisant `Iprobe` pour détecter la présence de messages sans bloquer l’exécution. Cette organisation permet de mesurer séparément le temps de rendu et le temps de calcul, et d’éviter que l’affichage ne ralentisse la simulation.